

Fret Ferret: Driving Educational Games with Contextual Multi-Armed Bandits to Accelerate Human Learning

Douglas Mason
douglas@koyotescience.com
Koyote Science, LLC
San Francisco, California, USA

Dan Cote
dan@koyotescience.com
Koyote Science, LLC
San Francisco, California, USA

ABSTRACT

This work introduces a robust software trainer for the guitar, called *Fret Ferret*, which utilizes the contextual multi-armed bandit class of algorithms, as well as implicit signals from the user, to customize lessons to each user automatically and in real time, avoiding the need to tweak a large selection of game parameters in order to obtain a desired difficulty level. We discuss the consequences of using these algorithms to drive new styles of computer-human interaction that can accelerate our learning process and help us reach new mental achievements. We elaborate on the details of our algorithms, how they inform our user interface design, and how we address the difficulty of scaling the development of a large number of machine learning models to adapt to many mini games and each user individually. Finally, we address future work that can further augment the performance and capabilities of our software.

CCS CONCEPTS

- Applied computing → Interactive learning environments;
- Computing methodologies → Learning from implicit feedback; Active learning settings.

KEYWORDS

machine learning, bandit algorithms, interface design

ACM Reference Format:

Douglas Mason and Dan Cote. 2022. Fret Ferret: Driving Educational Games with Contextual Multi-Armed Bandits to Accelerate Human Learning. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '22 Extended Abstracts)*, April 29-May 5, 2022, New Orleans, LA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3491101.3519907>

1 INTRODUCTION

Software for training guitar players has generally been limited to random selections of questions with a focus on correctness over speed[1–3]. However, users quickly get bored when such systems ask too many questions they know by heart, or ask questions that are outside of their abilities. To address this limitation, the common solution has been to offer a large variety of randomization parameters for the user to adjust in order to eliminate questions as they become easier to answer.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI '22 Extended Abstracts, April 29-May 5, 2022, New Orleans, LA, USA

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9156-6/22/04.

<https://doi.org/10.1145/3491101.3519907>

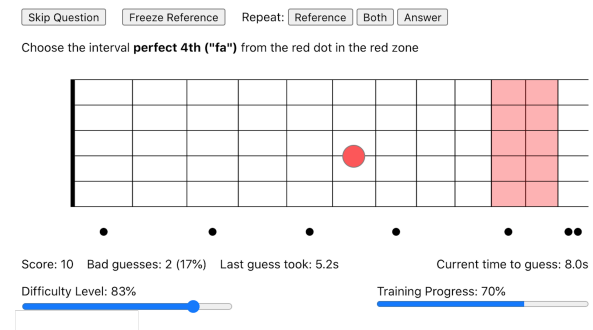


Figure 1: Example user interface for identifying intervals visually on the guitar fretboard, with the questions determined by a contextual multi-armed bandit aiming for moderate-to-high difficulty and that has been partially trained.

Here we introduce our own guitar training software *Fret Ferret* to show how we have marshaled a contextual multi-armed bandit algorithm[4, 9] to quickly learn from implicit user feedback during early trials. This allows us to hide the randomization parameters and automatically alter question selection to optimize for a desired difficulty level. For example, if a user desires high difficulty, the system will select questions which are likely to take the longest to answer and focus on their weak spots. If this proves too challenging to stay enjoyable, the user can reduce the difficulty by adjusting one intuitive parameter rather than many confusing ones (see Figures 1 & 2).

The multi-armed bandit class of algorithms were first introduced in the 1980s, but only recently have technological developments and improved methods allowed us to implement them easily and cheaply. In the last decade, they have exploded onto the education research scene[15, 19] as an ideal method for automatically tailoring questions to students without human intervention, which can greatly reduce labor costs and increase access. Beyond the literature we have examined on multi-armed bandits in educational games, our demonstration contributes the following:

- Our reward signal is based on **time-to-completion** instead of answer correctness.
- We **enhance exploration** using intuitive parameters that designers and users can adjust during gameplay.
- We combine state-of-the-art algorithms with **multiple modes of interactions** for a comprehensive educational experience

All of these contributions have been born out of necessity for building a large corpus of auto-personalizing games which can respond to the unique needs of advanced musical education.

2 WHY USE MULTI-ARMED BANDITS?

Multi-armed bandits are commonly used in commercial technology to optimize websites[21] as a replacement for A/B testing of different presentation options, such as choosing between two headlines for a newspaper article[6]. These algorithms automatically navigate the exploration/exploitation trade-off by first selecting options randomly (exploration) then honing in on the most promising ones (exploitation) in a manner that reduces the regret (the difference between real-world performance and ideal performance). Because of their data efficiency, bandits are ideal for personalization in situations where there are few opportunities to learn a user's preferences.

In our application, we use a multi-armed bandit to optimize for a desired difficulty level as evaluated implicitly by time-to-completion: high difficulty correlates to questions that take the longest to answer, and vice versa. Difficulty is measured as the percentile of random questions whose predicted time-to-completion are shorter than the measured time-to-completion for the given question. The bandit trains against the time-to-completion target, and questions are ranked according to the rank score. Given a prescribed difficulty level D , question Q with predicted time-to-completion $T(Q)$, and time-to-completion percentile $P(Q; Q)$ for Q among all questions Q , we write the rank score as

$$R(Q) = \text{abs}[P(Q; Q) - D] \quad (1)$$

Ranking is unaffected by additive shifts or multiplicative constants. Note that because of the additional restraint of matching a desired difficulty level, our approach does not match the bandit formalism strictly, except when D is set to 100%. Rather, we have decoupled the target that the bandits are trained against from the ranking procedure used to select questions, but the desired behavior remains the same.

We chose time-to-completion as a metric because it is difficult to cheat, making it one of the most reliable measures of cognition in the psychology literature, as highlighted by the Implicit Association Test[10]. Another reason is that our target audience of musicians desire not only to be correct, but to recall information as quickly as possible. Moreover, optimizing computer-human interaction to consistently operate at the edge of the user's abilities is the hallmark of creating "flow" states that are intrinsically enjoyable to the user and critical for creative development[7], while keeping users engaged and excited to keep using our system.

3 STRATEGY CONSIDERATIONS

Bandits are necessary in environments where data is expensive to collect – in our case, because each question requires time for the user to respond, and because users have limited patience. When a bandit makes predictions and acts on them with a given strategy, it not only samples the most promising options, it also learns the most about the domain of data where those options reside. Unlike traditional supervised machine learning models, which attempt to reflect the full domain of possible inputs in an unbiased fashion,

bandits are intentionally blind to unexplored regions in the input space since it is costly to obtain information about them. In our application, this behavior manifests itself when the user changes the desired difficulty level, since the bandit must re-engage exploration, and will make some wrong guesses as it learns about the new data domain, until it is confident it can exploit at the new difficulty level again.

There are many strategies available to automate this process, and we use Thompson sampling[5, 18], in which the prediction model is replaced by a distribution of models, one of which is sampled to perform predictions for each question that is asked. Model predictions, and their confidence intervals, are transformed by Equation 1 before being acted upon by any given strategy. We selected Thompson sampling because it only introduces one hyperparameter to tune, the number of prediction samples, which is generally set to 100, and it has shown stronger performance over epsilon-greedy and upper confidence bound (UCB) strategies in similar contexts[17].

For the prediction model, we use a custom implementation of Bayesian linear regression [12] (BLR), which provides us model distributions and Bayesian sampling to enable Thompson sampling. BLR has the benefit of only requiring $O(N_{\text{features}}^2)$ storage and $O(N_{\text{features}}^3)$ computation, where N_{features} is the number of features in the model. In addition to its speed and performance guarantees, BLR is an attractive solution because it only introduces one new hyperparameter to tune: the regularization constant, which can generally be left to a standard value such as 1/100th of the vector norm of the inputs after a set number of inputs have been observed.

With *Fret Ferret*, users specify a difficulty level between 0% and 100%, with the default set to 80%, and which is interpreted as a difficulty percentile of available questions. We found that only choosing the question at that exact percentile, which is the traditional bandit setup, led to poor experiences with insufficient variety to keep the user engaged, even though Thompson sampling partially randomizes its selection process anyways. Thus, we artificially increased the variety of our questions by selecting not only the most-closely-ranked question, but also others selected at random from a 10% population pool of questions with ranks close to the desired percentile. Anecdotal evaluations found this approach to be much more enjoyable and led to longer sessions.

4 MODES OF INTERACTION

The guitar trainer software we designed is a collection of many mini-games, each requiring a different prediction model to run its associated multi-armed bandit. The list of available games comes from the cross product of key skills we aim to train. On the one hand, players must be able to locate notes which are provided by name to gain a good *absolute* understanding of the fretboard, as well as musical intervals which can be written or played musically to gain a good *relative* understanding. On the other hand, we force users outside of their comfort zone by asking them to identify these locations under various restrictions which mimic common playing situations: (1) anywhere on the fretboard, (2) on a specified string, or (3) within a specified region of contiguous frets, usually 3 but other sizes are possible. Higher-level games include spelling out all

Available Intervals:

Types: ☐ All ☐ None ☐ 3rds & 6ths ☐ Major ☐ Minor

☐ U ☒ m2 ☒ M2 ☒ m3 ☒ M3 ☒ P4 ☒ TT ☒ P5

☒ m6 ☒ M6 ☒ m7 ☒ M7

Octaves: ☐ -3 ☐ -2 ☒ -1 ☐ 0 ☐ 1 ☐ 2 ☐ 3

Minimum Interval Distance: 0

Maximum Interval Distance: 17

Interval Group Length: 1

Available Fret Distances:

☐ All ☐ None ☒ Default ☐ Extremes

☐ end thru -7 ☐ -6 ☐ -5 ☒ -4 ☒ -3 ☒ -2 ☒ -1 ☒ 0

☒ 1 ☒ 2 ☒ 3 ☒ 4 ☒ 5 ☒ 6 ☒ 7 thru end

Available String Distances:

☐ All ☐ None ☒ Default ☐ Extremes

☐ -5 ☒ -4 ☒ -3 ☒ -2 ☒ -1 ☒ 0 ☒ 1 ☒ 2 ☒ 3 ☒ 4 ☐ 5

Available Answer Zone Sizes:

☐ 1 ☒ 2 ☐ 3

Available Answer Strings:

☒ All ☐ None

☒ 1 ☒ 2 ☒ 3 ☒ 4 ☒ 5 ☒ 6

Available Answer Frets:

☐ All ☐ None ☒ Without Open ☒ Middle 6

☐ 0 ☒ 1 ☒ 2 ☒ 3 ☒ 4 ☒ 5 ☒ 6 ☒ 7 ☒ 8 ☒ 9 ☒ 10 ☒ 11 ☒ 12

Available Answer Notes:

☒ All ☐ None ☒ Naturals ☒ Sharps ☒ Flats

☒ Ab ☒ A ☒ Bb ☒ B ☒ C ☒ C# ☒ D ☒ Db ☒ D# ☒ Eb ☒ E ☒ F ☒ F# ☒ Gb ☒ G ☒ G#

C Chromatic

Available Reference Note Strings:

☒ All ☐ None

☒ 1 ☒ 2 ☒ 3 ☒ 4 ☒ 5 ☒ 6

Available Reference Note Frets:

☐ All ☐ None ☒ Without Open ☒ Middle 6

☐ 0 ☒ 1 ☒ 2 ☒ 3 ☒ 4 ☒ 5 ☒ 6 ☒ 7 ☒ 8 ☒ 9 ☒ 10 ☒ 11 ☒ 12

Available Reference Notes:

☒ All ☐ None ☒ Naturals ☐ Accidentals

☒ A ☒ Bb ☒ B ☒ C ☒ Db ☒ D ☒ Eb ☒ E ☒ F ☒ F# ☒ G ☒ G# ☒ Ab

C Chromatic

Figure 2: Taken from actual gameplay during early development of *Fret Ferret*, we show the randomization parameters users otherwise need to adjust for the game in Figure 1 without the technology discussed in this article.

notes in a chord or scale, starting with either a root name or location indicated visually on the fretboard. At the highest level, a chord progression can be performed by our web app, with the corresponding notes shown on the app’s fretboard, while the performer improvises their own melodies which are also visually indicated on the same fretboard, to identify when the performer is playing “within” the chord and when they are playing “outside”. The chord progressions can be defined by the user and even include key changes, making this interaction mode ideal for developing jazz fluency.

We have provided a syllabus of 32 such games in increasing order of sophistication to help the user along their journey. While this breadth enables us to work with players possessing a wide range of education levels, the large number of mini games introduces the burden of engineering the features used to train the models that run the bandits powering each game. As more features are added – for example, not only encoding which fret the answer note is played on, but also the string – the model begins with greater uncertainty in its predictions, requiring more trials before it can refine those uncertainties. The bias-variance trade-off expresses itself in our games such that models with more features are more capable of finding diamonds in the rough, such as a particularly challenging fretboard location or note name for the user. Conversely, models with fewer features quickly pick out gestalt features, such as which strings or fretboard areas the user has difficulty with, but are incapable of refining their predictions further. We have tuned the feature sets through trial-and-error to find those that allow the model to learn as quickly as possible while still capturing the most important aspects for predicting difficulty.

Working with the field of music provides a wide variety of ways to interact with software, in part thanks to the proliferation of portable touch-screen devices with high-quality speakers and microphones that can even work simultaneously without overstepping each other. At the moment, *Fret Ferret* provides the following avenues for interaction:

- The user can click with a mouse or their finger on the fret position and string corresponding to their answer
- The user can play the corresponding note, either on their own guitar or any instrument of their choosing, even by humming the note

- The user can play the corresponding note on a MIDI instrument connected to the device

While working directly with the GUI is intuitive and always-available, the last two modes open up exciting possibilities with ear-training and transferring knowledge directly to the instrument, since the user can work with their native instrument instead of mapping one’s knowledge to a GUI. These modes also make *Fret Ferret* a viable trainer for any instrument, but without the added restrictions of forcing the user to play notes on a given string or fret region which are specific to the guitar. Future iterations may adapt the system to meet the needs of other stringed instruments.

With modern pitch detection software[11], accurate predictions of pitch can be made within a few hundred milliseconds. This means that while fast solos and runs are difficult to accommodate, moderate speed and rhythm can be maintained throughout the training session. Combining pitch detection with audio cues, an entirely-audio game can be played. Users of *Fret Ferret* have reported not just improved abilities at the guitar, but also improved listening skills that help the user identify the scale degrees (the “do-re-mi” label) of music that they listen to in real-time, helping them become better composers and improvisers. Pitch detection for multiple notes at once is an ongoing field of research[8], and could be used in the future to add the ability to play the notes of a chord simultaneously, rather than individually as is currently required.

5 CONCLUSIONS AND FUTURE WORK

The *Fret Ferret* guitar training software introduced here is unique for using implicit feedback and machine learning to tune its questions to the user automatically, radically simplifying the interface design so that only a single parameter needs to be controlled. This allows for easy adoption and a comfortable learning experience that eliminates wasted time as the student reaches towards their goals. By demonstrating the system and identifying key stumbling blocks, we hope to encourage others to create more educational tools driven by artificial intelligence that improve our human intelligence. There are, however, many ways we hope to improve our system and *Fret Ferret* further, which we discuss in greater length in our expanded white paper[14].

Fret Ferret is run by models that are unique to each mini game and user, and therefore its models start from scratch whenever they encounter a new user or a user begins a new game. But what if we could begin with head start by training a global federation of models for all users and each mini game? One way to accomplish this goal is to make a copy of each global model and fine-tune it to each user, in a process called transfer learning[16]. Another employs chaining uses predictions from the global model as an additional input to an associated personalized model, allowing the personalized model to learn a gate parameter that determines how much to weigh the contribution from the global model[14]. Because the gate parameter is interpretable, it gives the developer or user control over the influence of global predictions even as the bandit learns the ideal value, at the cost of optimizing the data collection process.

Our system optimizes individual questions based on the student’s response to them, but it does not consider how sequences of questions can lead to long-term goals. Work integrating reinforcement

learning techniques tuned to customer-facing applications[13] can help explore this avenue beyond the current literature applying these techniques in an educational setting[20].

Lastly, the interactive demonstration presented here has not been subjected to user experience studies, by which we can measure user progress and retention as a function of the parameters we use to define our system, such as using different feature engineering approaches, etc., and we would like to pursue this angle as resources allow. In the meantime, *Fret Ferret* is free and open to the public, along with our 32-course syllabus, and can be accessed at <http://fretferret.com>.

REFERENCES

- [1] 2020. *FaChords Guitar*. <https://www.fachords.com/guitar-learning-software/>
- [2] 2020. *Guitar Notes Fretboard Trainer*. <https://www.guitarorb.com/guitar-notes>
- [3] 2020. *Strong Apps*. <http://www.strongapplications.com>
- [4] Donald A. Berry and Bert Fristedt. 1985. *Bandit problems : sequential allocation of experiments* / Donald A. Berry, Bert Fristedt. Chapman and Hall London ; New York. viii, 275 p. : pages.
- [5] Olivier Chapelle and Lihong Li. 2011. An Empirical Evaluation of Thompson Sampling. In *Advances in Neural Information Processing Systems*, J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Q. Weinberger (Eds.), Vol. 24. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2011/file/e53a0a2978c28872a4505bdb51db06dc-Paper.pdf>
- [6] Anna Coenen. 2020. *How The New York Times is Experimenting with Recommendation Algorithms*. <https://open.nytimes.com/how-the-new-york-times-is-experimenting-with-recommendation-algorithms-562f78624d26>
- [7] Mihaly Csikszentmihalyi. 1991. *Flow: The Psychology of Optimal Experience*. Harper Perennial, New York, NY. http://www.amazon.com/gp/product/0060920432/ref=si3_rdr_bb_product/104-4616565-4570345
- [8] Anders Elowsson. 2018. Polyphonic Pitch Tracking with Deep Layered Learning. *CoRR* abs/1804.02918 (2018). arXiv:1804.02918 <http://arxiv.org/abs/1804.02918>
- [9] J. C. Gittins. 1989. *Multi-armed Bandit Allocation Indices*. Wiley, Chichester, NY.
- [10] Anthony G Greenwald, Debbie E McGhee, and Jordan L. K Schwartz. 1998. Measuring Individual Differences in Implicit Cognition: The Implicit Association Test. *Journal of personality and social psychology* 74, 6 (1998), 1464–1480.
- [11] Peter Hayes. 2020. *pitchfinder*. <https://github.com/peterkhayes/pitchfinder>
- [12] Douglas Mason. 2021. *Gaussian processes and equivalent Bayesian linear regressions*. <https://doi.org/10.5281/zenodo.5828879>
- [13] Douglas Mason. 2021. *Real-World Reinforcement Learning*. <https://doi.org/10.5281/zenodo.5828870>
- [14] Douglas Mason. 2022. *The Koyote Science, LLC, Approach to Personalization and Recommender Systems*. <https://doi.org/10.5281/zenodo.455689261>
- [15] John Mui, Fuhua Lin, and M Dewan. 2021. Multi-armed Bandit Algorithms for Adaptive Learning: A Survey. In *International Conference on Artificial Intelligence in Education*. Springer, 273–278.
- [16] Sinno Jialin Pan and Qiang Yang. 2009. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering* 22, 10 (2009), 1345–1359.
- [17] Daniel Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. 2020. A Tutorial on Thompson Sampling. arXiv:1707.02038 [cs.LG]
- [18] Steven L. Scott. 2010. A modern Bayesian look at the multi-armed bandit. *Applied Stochastic Models in Business and Industry* 26, 6 (2010), 639–658. <https://doi.org/10.1002/asmb.874> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/asmb.874>
- [19] Avi Segal, Yossi Ben David, Joseph Jay Williams, Kobi Gal, and Yaar Shalom. 2018. Combining difficulty ranking with multi-armed bandits to sequence educational content. In *International conference on artificial intelligence in education*. Springer, 317–321.
- [20] Adish Singla, Anna N Rafferty, Goran Radanovic, and Neil T Heffernan. 2021. Reinforcement Learning for Education: Opportunities and Challenges. *arXiv preprint arXiv:2107.08828* (2021).
- [21] J.M. White. 2012. *Bandit Algorithms for Website Optimization: Developing, Deploying, and Debugging*. O'Reilly Media. <https://books.google.com/books?id=xnAZLjqGybWC>